Mx-Modes
A Meta-Harness Framework for Multi-Mode AI Operation

Technical Architecture Reference

JD Longmire Northrop Grumman Fellow

https://orcid.org/0009-0009-1383-7698

Micah Longmire Sr. AI Architect

https://orcid.org/0009-0006-7608-9322

Abstract

Mx-Modes is a meta-harness framework that structures AI operation through explicit mode orientation. A root mode.md file activates a governing harness composed of five discipline-bearing surfaces — Mind, Morals, Mission, Memory, and Methods — plus the execution surface Means. These surfaces separate reasoning discipline, permission boundaries, operational purpose, contextual continuity, codified tradecraft, and execution capability. The framework treats the model as a probabilistic generator and the harness as the governing architecture that orients, constrains, and operationalizes model behavior.

The framework rests on one architectural claim: ***AI behavior should be oriented before it is executed.***

Most AI implementations begin with capability: models, tools, APIs, agents, plugins, automations, workflows, and integrations. Capability without orientation produces systems that can act before they have been properly governed. Mx-Modes reverses that order. It defines the operating posture first, then routes execution through governed means.

The architecture is designated MxM: a multi-mode meta-harness bracketed by the M-governed surfaces, where the x keeps the framework flexible and extensible. Mx-Modes is the framework name; MxM is the compact architectural designation used in diagrams, tables, and implementation references. The two name the same thing at different levels of formality.

LONGMIRE'S AI MAXIMS
1. Never trust the AI.
2. Overtake AI, or it will overtake you.
Maxim 1 is AI Zero-Trust Architecture (AI-ZTA): no model output is accepted ungoverned. J. Longmire

Scope and Non-Claims

Mx-Modes is a harness architecture. It governs how a model is oriented, constrained, and operationalized. It is not the following things, and reading it as any of them will measure it against the wrong standard:

- Not a model architecture. Mx-Modes governs a model; it does not specify how one is built or trained.
- Not a safety standard. It provides structure for governance but certifies nothing and asserts no safety guarantee.

- Not a compliance framework. It can carry compliance constraints within Morals, but it does not define them and does not satisfy any external mandate on its own.
- Not a replacement for enterprise policy. It is a structure into which policy is loaded, not a substitute for the policy itself.

What Mx-Modes provides is categorical separation and an explicit control structure. Its value is organizational clarity over model behavior, not a claim about model capability or assurance.

On separating Mind and Morals. The framework separates the documents that govern reasoning and permission, not the faculties themselves. In application these interpenetrate: sound reasoning already carries moral commitments, and applying a boundary requires reasoning about the case. The precedence model presumes exactly this collision. What Mx-Modes separates is the control surface, the inspectable specification of each, not the cognition that applies them. Governing reasoning and permission through distinct, auditable artifacts is the point, and it holds whether or not the underlying judgment is unified.

1 Framework Purpose

Surface
Governing question
Mind
How should the system reason?
Morals
What boundaries govern action?
Mission
What is the system trying to accomplish?
Memory
What continuity must be preserved?
Methods
How is the work done to a standard?
Means
What capabilities may be used, and how?

These questions must remain distinct. When they collapse into one another, the harness becomes unstable. The framework enforces a set of categorical non-identities:

- A tool is not permission.
- A memory is not mission.
- A persona is not authority.
- A fluent answer is not verified reasoning.
- A model output is not the same thing as judgment.

Mx-Modes holds these categories apart and places them in an explicit control structure.

2 Core Concept

Mx-Modes is built around a root file named mode.md, the entry point into the harness. It identifies the active operating mode, points to the meta-harness structure, establishes load order, and defines precedence rules. A basic repository structure:

```
/
├── mode.md
└── meta-harness/
├── mind.md
├── morals.md
├── mission.md
├── memory.md
└── means.md
```

The root file does not carry the whole system. It orients the system.

```
mode.md
|
|-- identifies the active mode
|-- defines the harness load order
|-- points to the meta-harness files
|-- establishes precedence rules
|-- governs conflict resolution
|
v
meta-harness/
```

The meta-harness contains five discipline-bearing surfaces plus the Means execution surface (six surfaces in total):

```
meta-harness/
|
|-- mind.md reasoning discipline
|-- morals.md permission boundaries
|-- mission.md purpose and objectives
|-- memory.md continuity and context
|-- methods.md tradecraft discipline
|-- means.md execution routing
```

The architecture is deliberately simple. Its value comes from categorical clarity, not from complexity.

2.1 What Is a Mode?

A mode is a declared operating posture that configures how the harness interprets purpose, permission, reasoning, memory, and execution for a given context. A mode may correspond to a task family, role, environment, risk posture, or operational state. It does not replace the meta-harness; it activates and contextualizes it.

Mode
Posture it selects
Research
Wide reasoning latitude, source discipline foremost, low execution authority.
Architecture
Structural reasoning, design-standard constraints, artifact generation enabled.
Red-team
Adversarial reasoning, expanded probing latitude, contained execution.
Artifact-generation
Output-quality standards foremost, format and citation constraints binding.
Compliance-review
Morals constraints foremost, conservative execution, mandatory traceability.
Mission-operator
Mission objectives foremost, full surface application, governed execution.
The same architecture, posture-selected per context, is what the framework means by multi-mode operation.

2.2 Lifecycle

A mode runs through a fixed operating sequence. The lifecycle makes explicit where orientation ends and execution begins, and where traceability is recorded.

1. Define the active mode.
2. Load the meta-harness surfaces.
3. Interpret the request through Mind, Morals, Mission, and Memory.
4. Route authorized execution through Means.
5. Record traceability for governed action.
6. Revise surfaces as operating conditions change.

Steps one through three are orientation; step four is execution; step five closes the loop the worked examples in Section 8 specify; step six returns control to the human who owns the harness.
2.3 Architecture at a Glance

The figure below shows the whole architecture on one page: the root orientation file, the five discipline-bearing surfaces with Means as the execution bridge (six surfaces in total), the two orderings that govern the surfaces, the means substructures, and the operating lifecycle with its revision loop.

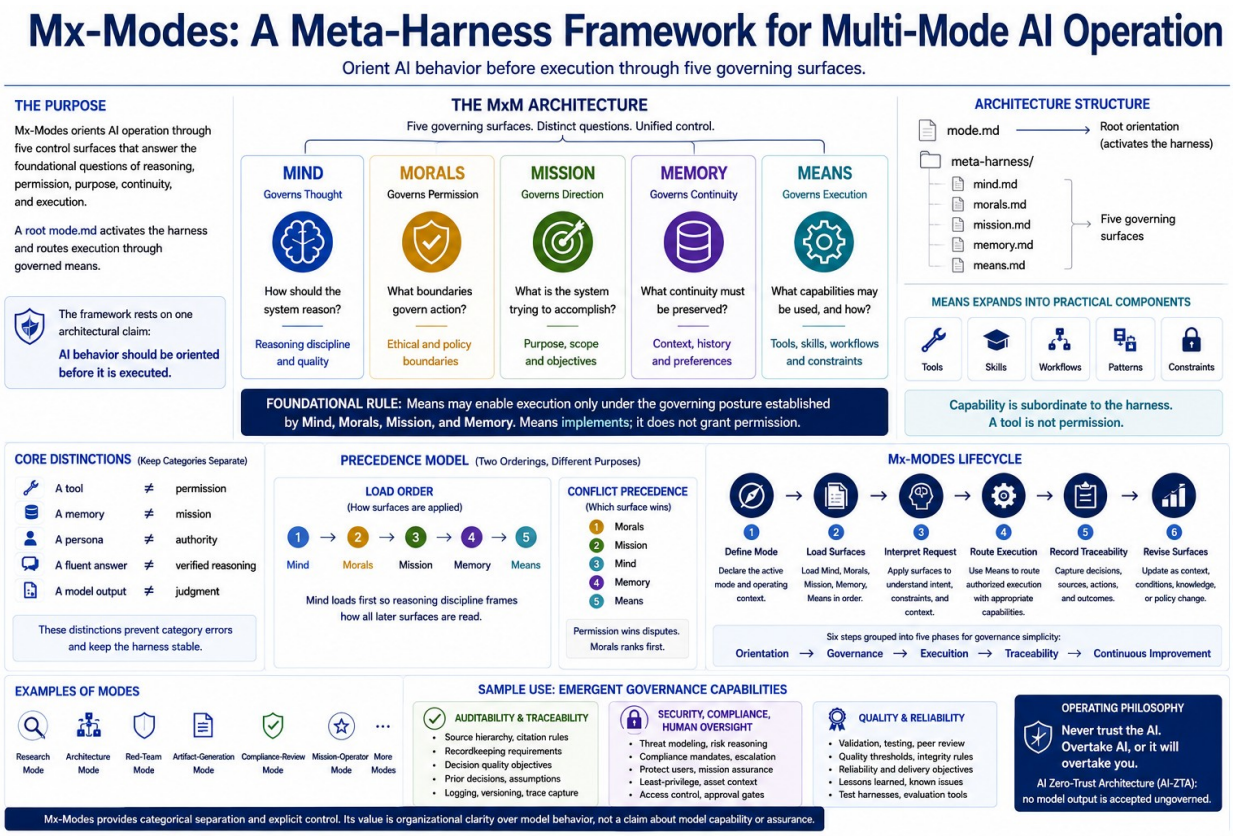

Figure 1. MxM architecture: the simplified governance view, showing surfaces, the two orderings, and the lifecycle. Surface interpenetration in application is discussed in the front matter.

3 The Six Surfaces (Five Disciplines + Execution)

3.1 Mind

mind.md governs reasoning discipline. It defines how the system should infer, evaluate, qualify, cite, challenge, and respond. This layer is where epistemic posture lives. It should include uncertainty handling, source discipline, anti-hallucination rules, logic patterns, confidence signaling, and quality thresholds.

Typical contents:

- inference rules
- uncertainty rules
- source hierarchy
- citation discipline
- reasoning patterns
- anti-hallucination posture
- distinction between fact, inference, speculation, and synthesis

Mind governs thought.

3.2 Morals

morals.md governs permission. It defines what the system may do, may not do, must refuse, must escalate, or must constrain. This layer may carry ethical, legal, organizational, contractual, security, or mission-specific boundaries.

Typical contents:

- ethical constraints
- refusal conditions
- user-protection rules
- data-handling boundaries
- authority limits
- escalation rules
- permitted and prohibited uses
- integrity principles

Morals governs permission.

3.3 Mission

mission.md governs direction. It defines what the system is trying to accomplish, preventing the system from treating every prompt as an isolated task detached from a larger operating purpose. Mission anchors the system in role, audience, desired outcome, scope, and success criteria.

Typical contents:

- mission statement
- supported use cases
- target users
- operating objectives
- success criteria
- scope boundaries
- deliverable expectations
- assumptions

Mission governs direction.

3.4 Memory

memory.md governs continuity. It defines what prior context matters, how project state should be preserved, and how previous decisions should inform future work. Memory should inform execution, but it should not redefine the mission or override permission boundaries.

Typical contents:

- user preferences
- project context
- prior decisions
- known assumptions
- terminology
- standing constraints
- continuity rules
- update and decay rules

Memory governs continuity.

3.5 Methods

methods.md governs tradecraft. It holds the codified, evidenced, reusable procedures and rigor disciplines for doing a kind of work to a standard. Where Mind governs how the system reasons, Morals what it may and must do, Mission what it pursues, and Memory what persists, Methods governs how the work is done well. It is the fifth discipline-bearing surface; with Means (execution) the harness presents six surfaces in total.

Methods recommends; it does not constrain, implement, or reason. This is the line that distinguishes it from its nearest neighbor, Morals: the boundary is enforcement and obligation. Morals holds the obligatory, enforced disciplines — violating one is a compliance breach. Methods holds recommended craft — departing from one is suboptimal, not a violation. A surgeon's ethics are not their surgical technique; both are real faculties, and conflating them loses something.

The surface is the entry point of a graduation pipeline through which a discipline hardens: **practice → method (codified) → moral (obligatory) → means (enforced)**. A repeated practice is codified as a method; a method that proves it must be mandatory graduates to a Morals obligation; an obligation that can be mechanized graduates again to a Means enforcement. A single discipline can therefore project across three surfaces simultaneously — the technique of a review is a Method, the duty to review before declaring done is a Moral, the pre-commit check is Means — while the large body of non-obligatory craft remains in Methods. This pipeline is the structural form of the principle that rigor is enforced by executable gates, not by documentation alone.

A harness deployed within a larger architecture will also encounter architectural patterns — cross-cutting shapes in how the system itself is composed. Methods (an MxM surface) sits at the operator altitude: how the agent does its work. Architectural patterns sit at the architecture altitude: how the system is shaped. They are peers at different altitudes; Methods may cite a pattern, but neither subsumes the other.

The surface admits a practice as a method only when it is evidenced (established through repetition, not asserted speculatively), named and reusable, and carries its rationale (the why, not only the what). This bar is what keeps the surface a disciplined faculty rather than a junk drawer.

Typical contents:

- codified procedures + best-practice captures
- rigor disciplines (QA, review, verification, validation techniques)
- technique catalogs proven through repetition
- research-capture protocols (survey before build)
- operator session-lifecycle disciplines
- the graduation pipeline (practice → method → moral → means) tracking
- citation links to architectural patterns (operator vs architecture altitude)

Methods governs tradecraft.

3.6 Means

means.md governs execution. This is the bridge from meta-harness governance into practical capability. It identifies the tools, skills, workflows, patterns, integrations, and artifact channels available to the system. Means should function as a routing layer, not a dumping ground.

Typical contents:

- available tools
- available skills
- workflows
- reusable patterns
- execution constraints
- artifact standards
- integration rules
- quality-control procedures

Means governs execution.

4 Means as the Practical Branching Layer

The means.md file is where the framework becomes operational. It answers what the harness can do, which substructure governs each action, what the preconditions are, what boundaries apply, and what quality checks are required before output. A recommended structure:

```
meta-harness/
└── means.md
│
├── tools/
│   ├── search.md
│   ├── code.md
│   ├── documents.md
│   ├── spreadsheets.md
│   ├── slides.md
│   ├── diagrams.md
│   └── integrations.md
│
├── skills/
│   ├── research.md
│   ├── writing.md
│   ├── analysis.md
│   ├── architecture.md
│   ├── evaluation.md
│   └── artifact-generation.md
│
├── workflows/
│   ├── briefing-development.md
│   ├── whitepaper-development.md
│   ├── source-validation.md
│   ├── executive-synthesis.md
│   ├── solution-architecture.md
│   └── red-team-review.md
│
├── patterns/
│   ├── rag.md
│   ├── agent-delegation.md
│   ├── human-in-the-loop.md
│   ├── human-on-the-loop.md
│   ├── approval-gates.md
```

```
│   ├── context-management.md
│   └── quality-control.md
│
└── constraints/
    ├── tool-boundaries.md
    ├── data-handling.md
    ├── citation-rules.md
    ├── security-rules.md
    └── artifact-standards.md
```

This gives the framework a clean expansion model. New tools, skills, workflows, and patterns can be added under means/ without rewriting the rest of the harness. The practical rule is straightforward:

means.md routes capability. It does not grant permission by itself.

Capability must remain subordinate to the higher-order harness layers.

5 Precedence Model

Mx-Modes requires explicit precedence rules. Without precedence, a harness is a loose collection of instructions. With precedence, it is an operating architecture. Two orderings govern the system, and they are deliberately different.

5.1 Load order

Step	File
1	meta-harness/mind.md
2	meta-harness/morals.md
3	meta-harness/mission.md
4	meta-harness/memory.md
5	meta-harness/means.md

5.2 Conflict precedence

Rank	Surface
1	Morals
2	Mission
3	Mind
4	Memory
5	Means

Note on the two orderings. Load order and conflict precedence are intentionally distinct: Mind loads first so that reasoning discipline frames how all later surfaces are read, whereas Morals ranks first in conflict resolution so that permission boundaries override every other surface when instructions collide. The two sequences answer different questions, when a surface is applied versus which surface wins, and should not be reconciled into a single list.

The precedence ranking produces several governing rules:

- Means may never override Morals.
- Memory may inform Mission but may not redefine it.
- Mind governs reasoning quality across all layers.
- Mission determines what the harness is trying to accomplish.
- Morals determines what the harness may or may not do.

The framework compresses to a single sequence:
Morals governs permission.
Mission governs direction.
Mind governs thought.
Memory governs continuity.
Means governs execution.
This order is intentionally conservative. Execution is powerful only after orientation has occurred. Tool access without governing purpose is a liability.

6 Reference: mode.md

Mode
This file is the root orientation layer for Mx-Modes.
Mx-Modes is a meta-harness framework for multi-mode AI
operation. It orients model behavior through six surfaces:
five disciplines (Mind, Morals, Mission, Memory, Methods)
plus Means (the execution surface).
Framework Name
Name: Mx-Modes

Architecture: MxM
Meaning: A multi-mode meta-harness architecture built around Mind, Morals, Mission, Memory, and Means. The x keeps the framework flexible and extensible.
Function
This file activates the operating mode and points to the governing meta-harness structure.
Load Order
1. meta-harness/mind.md
2. meta-harness/morals.md
3. meta-harness/mission.md
4. meta-harness/memory.md
5. meta-harness/means.md
Precedence
When instructions conflict, apply the following precedence:
1. Morals
2. Mission
3. Mind
4. Memory
5. Means
Operating Principle
The model is not the harness.
The model generates probable continuations.
The harness constrains, directs, evaluates, and operationalizes those continuations.
Control Principle
AI behavior should be oriented before it is executed.
Execution follows orientation.
Capability follows permission.
Tool use follows purpose.
7 Reference: means.md

Means
means.md defines the practical capability layer of Mx-Modes.
It maps approved capabilities to their governing substructures.
Function
Means governs execution.
It identifies the tools, skills, workflows, patterns, and constraints available to the harness when performing work.
Means does not define the mission.
Means does not override moral constraints.
Means does not alter reasoning discipline.
Means does not treat tool access as permission.
Capability Branches
Tools
Tools are executable or callable capabilities.
See:
- means/tools/search.md
- means/tools/code.md
- means/tools/documents.md
- means/tools/spreadsheets.md

- means/tools/slides.md
- means/tools/diagrams.md
- means/tools/integrations.md
Skills
Skills are reusable human-directed work patterns.
See:
- means/skills/research.md
- means/skills/writing.md
- means/skills/analysis.md
- means/skills/architecture.md
- means/skills/evaluation.md
- means/skills/artifact-generation.md
Workflows
Workflows are ordered execution sequences.
See:
- means/workflows/briefing-development.md
- means/workflows/whitepaper-development.md
- means/workflows/source-validation.md
- means/workflows/executive-synthesis.md
- means/workflows/solution-architecture.md
- means/workflows/red-team-review.md
Patterns
Patterns are reusable architectural or behavioral templates.
See:
- means/patterns/rag.md
- means/patterns/agent-delegation.md
- means/patterns/human-in-the-loop.md (blocking; default halt)
- means/patterns/human-on-the-loop.md (supervisory; default proceed)
- means/patterns/approval-gates.md
- means/patterns/context-management.md
- means/patterns/quality-control.md
Constraints
Constraints define execution boundaries.
See:
- means/constraints/tool-boundaries.md
- means/constraints/data-handling.md
- means/constraints/citation-rules.md
- means/constraints/security-rules.md
- means/constraints/artifact-standards.md
Operating Rule
Means may enable execution only under the governing posture
established by Mind, Morals, Mission, and Memory.
Execution follows orientation.
Capability follows permission.
Tool use follows purpose.
Audit Traceability (Realization)
Means realizes the audit trail that the other surfaces
specify. Means implements; it does not author the standard.
- constraints/artifact-standards.md
Every governed action records its initiating surface and
the precedence decision that admitted it.

(Realizes the Morals requirement.)
- patterns/quality-control.md
No output is emitted until its trace satisfies the
acyclicity and source-discipline standard set by Mind.
(Realizes the Mind systematics.)
- constraints/data-handling.md
Trace records inherit the same handling boundaries as the
data they describe. A trace of a sensitive action may be
as sensitive as the action.
(Realizes Morals data-handling boundaries.)
8 Architectural Value

Mx-Modes provides five architectural benefits.

Separation of concerns. Reasoning, permission, mission, memory, tradecraft, and execution are governed by different harness surfaces, which makes the system easier to inspect, tune, and govern.

Scalability. New tools, workflows, integrations, artifact types, and execution patterns can be added under means/ without rewriting the framework.

Auditability. A reviewer can determine whether a behavior originated in the reasoning, moral, mission, memory, or means layer.

Portability. The same structure supports general-purpose assistants, enterprise copilots, agentic workflows, software development agents, research copilots, writing systems, and mission-specific AI operators.

Control discipline. The model is treated as a probabilistic generator operating inside a structured harness, not as an autonomous reasoner.

8.1 Worked Example: Auditability Is Distributed

Surface
Contribution to the audit trail
Morals
Requirement and accountability. Traces must exist and must be attributable; escalation and authority rules define accountability when an action is wrong.
Mission
Objective. For a given deployment, an audit trail is named as a success criterion when the mission calls for it.
Mind

Systematics. Defines what a valid trace is: the acyclicity and source-discipline standard an attribution must satisfy to count.

Means

Mechanism. Realizes the log that records which surface fired and which precedence decision admitted the action.

The direction is one way. Morals requires it, Mission targets it, Mind defines what valid looks like, and Means builds the artifact that satisfies all three. Means implements the standard; it does not author it. If Means were to define its own trace-validity standard, it would absorb Mind, repeating at the execution layer the same surface-collapse the framework prohibits everywhere else.

This example is the framework demonstrating its own thesis. No single surface produces auditability. It emerges from the separation. That is precisely the categorical clarity the framework claims as its central contribution, shown rather than asserted.

8.2 Worked Example: Security, Compliance, and Human Oversight

Surface

Contribution to security

Morals

Boundary. What may touch controlled data, where inference may run, what may cross the air gap. A permission constraint (means/constraints/security-rules.md).

Mind

Posture. Whether a configuration is actually secure, whether a threat model holds, what the blast radius of a failure is. An act of reasoning, not permission.

Means

Enforcement. The control that blocks the action or refuses the egress. The realization of the boundary, not its definition.

The hard case is a conflict between two things that both look like Morals: a compliance mandate and a security posture. A STIG may require a configuration that a threat model judges to raise real risk. The framework resolves this without letting the system override a mandate on its own reasoning.

Posture

Behavior and when it applies

Human-on-the-loop

Supervisory; default proceed. The system complies with the standing mandate automatically while a human supervises and may intervene. Used when residual risk is within tolerance. Blocking every routine compliance action on human approval would be operationally untenable.
Human-in-the-loop
Blocking; default halt. The action stops and waits for a human to accept the risk, seek a waiver, or alter the deployment. Used when Mind judges residual risk to exceed tolerance. The human is a required step in the execution path, not an observer of it.
The chain is therefore: compliance binds at Morals, Mind assesses residual risk, and the risk grade selects the gate. Within tolerance, human-on-the-loop proceeds under supervision. Beyond tolerance, human-in-the-loop halts and escalates to human authority through the escalation rules Morals already carries. Means realizes whichever gate fires; it does not choose which gate is warranted.

One dependency to name honestly. This resolution assumes that compliance from an external authority is not something internal reasoning may override. That reading is correct for defense IT, but it is a deliberate choice, not a result the precedence model forces. A different reading could argue that Mind, which loads first, should frame the compliance constraint rather than defer to it. The conservative reading is adopted here because in this domain a mandate is binding regardless of what any model concludes about it.

8.3 The General Property

Auditability, security, and human oversight share a structure. None is owned by a single surface. Each is specified by the upper surfaces and realized by Means: Morals states the requirement or boundary, Mind sets the standard or assesses the posture, Mission names the objective where one applies, and Means builds the mechanism that satisfies them. Means implements; it never authors. Three instances are enough to state the property in general terms.

Cross-cutting governance capabilities are emergent across the separation, not features of any one surface. The framework does not have an audit surface, a security surface, or an oversight surface, and it should not. Each capability is produced by the surfaces doing their distinct jobs and refusing to absorb one another. This is the separation-of-concerns thesis carried to its operational conclusion.

9 Enterprise AI Relevance

In enterprise AI, the most dangerous failure mode is often not model weakness but uncontrolled capability. When an AI system can retrieve data, call tools, draft artifacts, invoke workflows, interact with users, or trigger automations, the enterprise must govern more than output quality. It must govern action.

Surface
Enterprise mapping
Mind
Reasoning standards, source discipline, uncertainty handling
Morals
Policy and ethics; compliance mandates (FedRAMP, CMMC, ITAR/EAR, STIGs, ATO) as externally sourced permission; security boundaries. See Section 8.2 for how security posture and compliance are kept distinct.
Mission
Business purpose, operational objective, user outcome
Memory
Enterprise context, project history, user preferences, retained decisions
Means
Approved tools, APIs, workflows, automations, artifact channels
This makes Mx-Modes especially useful for agentic AI. As systems move from chat toward execution, the harness must clarify what the system is, what it is doing, what it remembers, what it may do, and which means are authorized.

10 Minimal Framework Statement

Mx-Modes is a meta-harness framework for multi-mode AI operation.
Mind governs thought.
Morals governs permission.
Mission governs direction.
Memory governs continuity.
Means governs execution.
mode.md activates the harness.
The meta-harness orients the model.
means.md routes practical capability.
The model generates.
The harness governs.
11 Conclusion

Mx-Modes is a disciplined way to structure AI behavior before execution. It treats the model as a generative substrate and the harness as the governing architecture around that substrate. Its main contribution is categorical clarity:

- Reasoning is not morality.
- Morality is not mission.
- Mission is not memory.
- Memory is not means.
- Means is not permission.

That clarity matters more as AI systems become more capable. The question will not be whether an AI system can act; many already can. The sharper question is whether its action is properly oriented, bounded, directed, informed, and governed. Mx-Modes provides a compact architecture for that work.
Disclaimer

Mx-Modes is independent, unaffiliated research. The views and framework presented here are those of the authors alone and do not represent the views, positions, or endorsement of

Northrop Grumman Corporation or any other employer or organization. This document is not a Northrop Grumman work product, deliverable, or proprietary asset, and the authors' institutional titles are provided for identification only. Nothing herein should be construed as an official statement of any organization with which the authors are affiliated.